📊 Monitorización con Prometheus y Grafana



📌 Descripción del Proyecto

Guía completa y profesional para el despliegue, configuración y administración de una infraestructura moderna de **monitorización de sistemas, servicios y bases de datos** mediante la pila **Prometheus + Grafana**, diseñada para el ciclo formativo de **Administración de Sistemas Informáticos en Red (2º ASIR)**.

A lo largo de este repositorio se aborda desde la teoría base de las bases de datos de series temporales (TSDB) y los *exporters*, hasta la monitorización de múltiples servidores Linux distribuidos, pruebas de estrés de hardware, diseño de dashboards personalizados en PromQL, alertas automáticas por correo electrónico (Gmail) y la monitorización de servidores de bases de datos MariaDB.

🏗️ Arquitectura General del Sistema

[Diagrama de flujo de arquitectura disponible en la versión web de GitHub]

📑 Índice de Contenidos

Unidad	Título del Módulo	Descripción y Objetivos	Acceso directo
00	Introducción a Prometheus y Grafana	Conceptos clave, TSDB, modelo Pull vs Push, anatomía de un Exporter y tipos de métricas.	📖 Ver Unidad 0
01	Instalación de Prometheus	Despliegue con binarios oficiales, usuario de sistema, servicio systemd y verificación web (`:9090`).	📖 Ver Unidad 1
02	Instalación de Grafana	Configuración de repositorios oficiales APT de Grafana OSS, demonio `grafana-server` y primer acceso (`:3000`).	📖 Ver Unidad 2
03	Añadir Prometheus como Data Source	Conexión e integración de Prometheus en Grafana, validación de la API y primer dashboard.	📖 Ver Unidad 3
04	Monitorización de varios servidores	Despliegue de Node Exporter en nodos remotos, configuración de `targets` en `prometheus.yml` y dashboard 1860.	📖 Ver Unidad 4
05	Estresando al servidor	Simulación de carga artificial de CPU, RAM e I/O de disco con `stress` y `stress-ng` y análisis de impacto en vivo.	📖 Ver Unidad 5
06	Consultas PromQL y Diseño de Paneles	Creación de consultas avanzadas con `rate()` , agregaciones, variables de plantilla y diseño de dashboards desde cero.	📖 Ver Unidad 6
07	Alertas con Gmail	Configuración de contraseñas de aplicación de Google, servidor SMTP en Grafana y reglas de alerta para incidentes.	📖 Ver Unidad 7
08	Tarea: Monitorización de MariaDB	Instalación de `mysqld_exporter` , usuario con permisos de métricas en MariaDB, integración y panel de control.	📖 Ver Unidad 8

🔌 Resumen de Puertos y Servicios Clave

Servicio / Componente	Puerto por defecto	Endpoint de Métricas	Propósito
Prometheus Server	`9090`	`/metrics`	Motor de recolección de métricas y consultas PromQL.
Grafana	`3000`	N/A	Interfaz visual interactiva y generación de alertas.
Node Exporter	`9100`	`/metrics`	Agente del sistema operativo (CPU, RAM, discos, red).
MySQL / MariaDB Exporter	`9104`	`/metrics`	Agente de rendimiento para el motor de base de datos.
Alertmanager	`9093`	`/metrics`	Gestor y enrutador de alertas de Prometheus (opcional).

📂 Estructura del Repositorio

```
.
├── 0.Introduccion.md                       # Teoría base, TSDB y conceptos previos
├── 1.Instalacion_Prometheus.md             # Instalación y servicio de Prometheus
├── 2.Instalacion_Grafana.md                # Instalación y servicio de Grafana
├── 3.Prometheus_Data_Source.md             # Configuración de origen de datos en Grafana
├── 4.Monitorizacion_Varios_Servidores.md   # Node Exporter y entornos multiservidor
├── 5.Estresando_al_servidor.md             # Pruebas de carga con stress y análisis
├── 6.Consultas_y_Paneles.md                # PromQL y creación de dashboards
├── 7.Alertas_con_Gmail.md                  # Sistema de alertas y notificaciones SMTP
├── 8.Tarea_Monitorizacion_MariaDB.md       # Práctica de monitorización de MariaDB
├── img/                                    # Capturas de pantalla y diagramas
├── LICENSE                                 # Licencia GNU General Public License v3.0
└── README.md                               # Portada y documentación general
```

👤 Autor

- **Daniel Silva** (@JohnDSil)
- 2º ASIR - Administración de Sistemas Informáticos en Red

📄 Licencia

Este proyecto está distribuido bajo la licencia GNU General Public License v3.0.

🚀 00. Introducción a Prometheus y Grafana

📋 1. ¿Qué es la monitorización moderna?

En la administración de sistemas tradicional, la monitorización solía basarse en sondeos binarios (*"¿el servidor responde al ping?"*, *"¿el puerto 80 está abierto?"*). En entornos modernos de producción, microservicios y **DevOps**, esto resulta insuficiente: es necesario comprender el comportamiento interno del sistema en el tiempo (saturación de CPU, consumo de memoria, latencias de peticiones, colas de disco o consultas lentas de bases de datos).

La combinación de **Prometheus** (como motor de recolección y base de datos de métricas) y **Grafana** (como plataforma interactiva de visualización y alertas) constituye el estándar de la industria (*de facto*) para la observabilidad en Linux y contenedores.

---

2. ¿Qué es Prometheus?

Prometheus es un sistema de monitorización y alertas de código abierto (*open source*) diseñado originalmente por SoundCloud y mantenido actualmente por la **CNCF** (*Cloud Native Computing Foundation*).

⚙️ Características principales

- **Almacenamiento de series temporales (TSDB):** Los datos se almacenan como flujos de valores marcados en el tiempo (*timestamps*) identificados por un nombre de métrica y pares clave-valor de etiquetas (*labels*).
- **Modelo Pull (Scraping):** En lugar de que los agentes envíen datos de forma masiva al servidor central (*Push*), Prometheus realiza peticiones HTTP periódicas a los destinos (*Pull*) para consultar sus métricas.
- **Lenguaje de consultas PromQL (*Prometheus Query Language*):** Permite filtrar, agregar, calcular tasas de cambio (*rates*) y proyectar tendencias en tiempo real.
- **Alertas y notificaciones configurables:** Detección de anomalías y derivación a *Alertmanager*.
- **Extensibilidad mediante *Exporters*:** Capacidad para monitorizar cualquier componente (servidores, redes, bases de datos, proxies) sin necesidad de modificar el código de la aplicación.

```
[Diagrama de flujo de arquitectura disponible en la versión web de GitHub]
```

---

3. ¿Qué son los Exporters?

Los **exporters** son programas ligeros o servicios que actúan como "traductores": leen las estadísticas internas de un sistema operativo, aplicación o base de datos y las exponen en un formato de texto plano comprensible por Prometheus a través de un endpoint HTTP (por defecto en la ruta `/metrics`).

📌 Exporters más utilizados

Exporter	Función	Puerto Habitual
Node Exporter	Recolecta métricas de hardware y del kernel de Linux (CPU, RAM, discos, red).	`9100`
MySQL / MariaDB Exporter	Expone estadísticas del motor relacional (conexiones, consultas, buffers, bloqueos).	`9104`
PostgreSQL Exporter	Métricas de rendimiento, transacciones y bloqueos de PostgreSQL.	`9187`
Blackbox Exporter	Prueba la disponibilidad y latencia de servicios de red (HTTP, HTTPS, DNS, TCP, ICMP).	`9115`
Nginx / Apache Exporter	Extrae estadísticas de peticiones concurrentes, tráfico y códigos HTTP.	`9113` / `9117`

🔍 Métricas extraídas por el Node Exporter:

- 🧠 **CPU:** Tiempos de procesamiento desglosados (`user`, `system`, `idle`, `iowait`, `steal`).
- 💾 **Memoria (`meminfo`):** Memoria RAM física total, libre, buffers, caché y memoria disponible real.
- 💽 **Discos (`diskstats`):** Lecturas/escrituras por segundo (IOPS), tiempos de espera (*await*) y rendimiento.
- 🌐 **Red (`netdev`):** Bytes recibidos/transmitidos, paquetes erróneos y caídos (*drop*).
- 📁 **Sistemas de archivos (`filesystem`):** Espacio total, ocupado e inodos en cada partición montada.
- ⚙️ **Carga del sistema (`loadavg`):** Promedio de carga a 1, 5 y 15 minutos.
- 🔒 **Entropía (`entropy`):** Disponibilidad de aleatoriedad del sistema para operaciones criptográficas.
- 🌡️ **Sensores (`hwmon`):** Temperaturas, voltajes y ventiladores.

---

4. ¿Qué es Grafana?

Grafana es la herramienta de visualización y análisis de datos de código abierto más popular del mundo. Permite transformar datos crudos procedentes de múltiples fuentes en cuadros de mando interactivos (*dashboards*) claros y estéticamente atractivos.

🌟 Puntos fuertes de Grafana:

- **Visualización avanzada:** Gráficos de series temporales, indicadores de estado (*Stat*), velocímetros (*Gauge*), mapas de calor (*Heatmap*), gráficos de barras y tablas interactivas.
- **Soporte multiorigen (*Data Sources*):** No solo se conecta a Prometheus; permite unificar datos de MySQL, PostgreSQL, InfluxDB, Elasticsearch, CloudWatch y decenas de fuentes en un único panel.
- **Dashboards públicos comunitarios:** Miles de plantillas prediseñadas listas para importar (por ejemplo, el ID `1860` para Node Exporter).

- **Sistema de alertas multicanal:** Notificaciones automáticas a través de correo electrónico (Gmail/SMTP), Telegram, Discord, Slack o webhooks cuando un valor sobrepasa un umbral crítico.

---

5. Conceptos clave antes de la instalación

Para comprender el funcionamiento de la infraestructura, es indispensable dominar los siguientes términos:

1. Métricas y Series Temporales

Una serie temporal en Prometheus está compuesta por: - **Nombre de la métrica:** Define el recurso medido (ej. `node_cpu_seconds_total`). - **Etiquetas (*Labels*):** Pares clave-valor que añaden dimensionalidad (ej. `{mode="idle", cpu="0", instance="172.26.12.21:9100"}`). - **Timestamp:** Marca temporal en milisegundos. - **Valor:** Un número flotante de 64 bits (*float64*).

2. Scraping, Exporter, Endpoint y Target

```
[Diagrama de flujo de arquitectura disponible en la versión web de GitHub]
```

- **Scraping:** Acción periódica en la que Prometheus hace una petición HTTP GET al endpoint para descargar las métricas.
- **Endpoint:** La URL exacta donde se sirven las métricas (por ejemplo, `http://192.168.1.10:9100/metrics`).
- **Target:** La instancia definida en la directiva de configuración de Prometheus (ej. `"192.168.1.10:9100"`).

❓ ¿Qué ocurre si tenemos dos exporters en el mismo servidor?

Si en un servidor Debian (`192.168.1.10`) se instalan Node Exporter y MySQL Exporter: - **Node Exporter:** Target `192.168.1.10:9100` $\rightarrow$ Endpoint `http://192.168.1.10:9100/metrics` - **MySQL Exporter:** Target `192.168.1.10:9104` $\rightarrow$ Endpoint `http://192.168.1.10:9104/metrics`

> **Resultado:** Tenemos un solo servidor físico/virtual, pero **dos targets y dos endpoints independientes** configurados en Prometheus.

---

6. Escenarios de Despliegue

A lo largo de este proyecto trabajaremos sobre los dos escenarios habituales:

🖥️ Escenario 1: Monomáquina (Entorno de pruebas / Laboratorio)

- **Objetivo:** Instalar Prometheus, Node Exporter y Grafana en un único equipo local.
- Prometheus: `http://localhost:9090`
- Node Exporter: `http://localhost:9100/metrics`
- Grafana: `http://localhost:3000`
- **Ideal para:** Demostraciones, desarrollo de paneles y pruebas iniciales.

🌐 Escenario 2: Entorno Distribuido (Producción)

- **Objetivo:** Un servidor central alojando Prometheus y Grafana, recolectando métricas a través de la red local desde múltiples servidores remotos (que ejecutan Node Exporter o MariaDB Exporter).
- **Ideal para:** Monitorización centralizada de infraestructuras empresariales.

⚙️ 01. Instalación y Configuración de Prometheus

📋 Introducción

Prometheus es una aplicación escrita en **Go** que se distribuye como un binario único autocontenido, lo que facilita enormemente su despliegue y mantenimiento en sistemas GNU/Linux.

En esta unidad detallamos el procedimiento estándar de instalación mediante **binarios oficiales** (el recomendado en entornos profesionales para contar con la versión más reciente) y la alternativa mediante el gestor de paquetes **APT**.

1. Método Recomendado: Instalación mediante Binarios Oficiales

Paso 1: Creación del usuario y directorios del sistema

Por motivos de seguridad, Prometheus nunca debe ejecutarse como usuario `root`. Creamos un usuario de sistema dedicado sin directorio personal ni acceso a shell interactiva:

```
# Crear grupo y usuario de sistema sin inicio de sesión
sudo groupadd --system prometheus
sudo useradd -s /sbin/nologin --system -g prometheus prometheus

# Crear directorios para configuración y base de datos TSDB
sudo mkdir -p /etc/prometheus
sudo mkdir -p /var/lib/prometheus
```

Paso 2: Descarga y descompresión de Prometheus

Accedemos al directorio temporal y descargamos la versión estable más reciente desde el repositorio oficial de GitHub de Prometheus:

```
cd /tmp
# Descargar el tarball oficial para arquitectura AMD64
wget https://github.com/prometheus/prometheus/releases/download/v2.51.2/prometheus-2.51.2.linux-amd64.tar.gz

# Descomprimir el archivo
tar -xvzf prometheus-2.51.2.linux-amd64.tar.gz
cd prometheus-2.51.2.linux-amd64
```

Paso 3: Instalación de binarios y bibliotecas de consola

Movemos los ejecutables al directorio estándar `/usr/local/bin/` y las librerías a `/etc/prometheus`:

```
# Copiar binarios principales
sudo cp prometheus /usr/local/bin/
sudo cp promtool /usr/local/bin/

# Copiar bibliotecas de consolas web clásicas
sudo cp -r consoles /etc/prometheus/
sudo cp -r console_libraries /etc/prometheus/
```

Verificamos que los ejecutables son reconocidos:

```
prometheus --version
promtool --version
```

Paso 4: Creación del archivo de configuración `prometheus.yml`

Creamos el archivo de configuración principal en `/etc/prometheus/prometheus.yml`:

```
sudo nano /etc/prometheus/prometheus.yml
```

Añadimos la siguiente configuración base:

```
# Configuración global
global:
  scrape_interval: 15s     # Frecuencia con la que se consultan los endpoints
  evaluation_interval: 15s # Frecuencia con la que se evalúan las reglas de alerta

# Configuración de los jobs de recolección (Scraping)
scrape_configs:
  # Monitorización del propio servidor Prometheus
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
```

Ajustamos los permisos para que pertenezcan al usuario `prometheus`:

```
sudo chown -R prometheus:prometheus /etc/prometheus
sudo chown -R prometheus:prometheus /var/lib/prometheus
sudo chmod -R 775 /etc/prometheus
sudo chmod -R 775 /var/lib/prometheus
```

💡 **TIP:** > Puedes validar la sintaxis del archivo de configuración antes de arrancar el servicio usando la herramienta oficial `promtool`: > ```bash > promtool check config /etc/prometheus/prometheus.yml > # Salida esperada: SUCCESS: /etc/prometheus/prometheus.yml is valid prometheus config file syntax > ```

Paso 5: Creación del servicio Systemd

Para gestionar Prometheus como un servicio en segundo plano que se inicie automáticamente con el sistema operativo, creamos la unidad de servicio:

```
sudo nano /etc/systemd/system/prometheus.service
```

Insertamos la siguiente definición:

```
[Unit]
Description=Prometheus Time Series Collection and Alerting Server
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
    --config.file=/etc/prometheus/prometheus.yml \
    --storage.tsdb.path=/var/lib/prometheus/ \
    --web.console.templates=/etc/prometheus/consoles \
    --web.console.libraries=/etc/prometheus/console_libraries \
    --web.listen-address=0.0.0.0:9090 \
    --web.enable-lifecycle

Restart=always

[Install]
WantedBy=multi-user.target
```

ℹ️ **NOTE:** > El flag `--web.enable-lifecycle` permite recargar la configuración de Prometheus en caliente sin reiniciar el servicio mediante una llamada HTTP: > `curl -X POST http://localhost:9090/-/reload`

Paso 6: Inicio y habilitación del servicio

```
# Recargar systemd para detectar el nuevo servicio
sudo systemctl daemon-reload

# Habilitar e iniciar Prometheus
sudo systemctl enable --now prometheus

# Comprobar el estado del servicio
sudo systemctl status prometheus
```

---

2. Método Alternativo: Instalación rápida vía APT

En sistemas Debian y Ubuntu, Prometheus también se encuentra en los repositorios oficiales de la distribución:

```
sudo apt update
sudo apt install -y prometheus
```

- **Ventaja:** Instalación en un único comando y servicio systemd configurado automáticamente.
- **Desventaja:** Las versiones de los repositorios de la distribución suelen estar varios meses por detrás de la versión oficial más reciente.

---

3. Comprobación y Verificación

1. Comprobación del puerto 9090

Verificamos que el puerto `9090` está en estado `LISTEN`:

```
ss -plunt | grep 9090
```

Salida esperada:

```
tcp   LISTEN 0      4096         *:9090            *:*    users:(("prometheus",pid=12345,fd=7))
```

2. Acceso a la Interfaz Web de Prometheus

Abre tu navegador web e ingresa a:

```
http://localhost:9090  (o http://<IP_DE_TU_SERVIDOR>:9090)
```

En la barra superior de navegación dispones de: - **`Graph` / `Table` :** Consola para ejecutar consultas PromQL en vivo y visualizar gráficos interactivos. - **`Status` $\rightarrow$ `Targets` :** Lista de todos los endpoints configurados con su estado (`UP` o `DOWN`), último scrape y duración. - **`Status` $\rightarrow$ `Configuration` :** Muestra el archivo `prometheus.yml` activo cargado en memoria.

Consulta de prueba inicial:

En el buscador de la pestaña `Graph`, escribe:

```
up
```

Pulsa el botón **Execute**. Deberás ver la métrica con valor `1`, confirmando que el propio Prometheus se está scrapeando con éxito.

📈 02. Instalación y Configuración de Grafana

📋 Introducción

Grafana es el motor visual encargado de transformar las métricas crudas almacenadas en Prometheus en paneles gráficos, tablas y cuadros de mando interactivos de alto rendimiento.

En esta unidad instalaremos la edición **Grafana OSS** (*Open Source Software*) a través del repositorio APT oficial de Grafana Labs, asegurando actualizaciones y estabilidad continua en Debian / Ubuntu.

1. Procedimiento de Instalación en Debian / Ubuntu

Paso 1: Instalación de dependencias previas

Nos aseguramos de que el sistema cuenta con las utilidades necesarias para gestionar repositorios seguros HTTPS y claves criptográficas:

```
sudo apt update
sudo apt install -y apt-transport-https software-properties-common wget gpg
```

Paso 2: Importar la clave GPG del repositorio oficial

Creamos el directorio seguro para los llaveros de APT (si no existe) y descargamos la clave pública oficial de Grafana Labs:

```
# Crear directorio de claves APT
sudo mkdir -p /etc/apt/keyrings/

# Descargar y convertir la clave GPG oficial
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null
```

Paso 3: Añadir el repositorio oficial estable de Grafana

Añadimos la línea del repositorio apuntando a la clave descargada:

```
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee /etc/apt/sources.list.d/grafana.list
```

Paso 4: Instalar Grafana OSS

Actualizamos la lista de paquetes para indexar el nuevo repositorio e instalamos el paquete `grafana`:

```
sudo apt update
sudo apt install -y grafana
```

Verificamos la versión instalada:

```
grafana-server -v
```

2. Gestión del Servicio con Systemd

Grafana incluye por defecto la unidad de servicio `grafana-server.service`. Para habilitarla al arranque e iniciarla inmediatamente:

```
# Recargar systemd
sudo systemctl daemon-reload

# Habilitar e iniciar el servicio grafana-server
sudo systemctl enable --now grafana-server

# Verificar el estado del servicio
sudo systemctl status grafana-server
```

Comprobación del puerto de red `3000`:

Verificamos que el servidor está escuchando en el puerto TCP `3000`:

```
ss -plunt | grep 3000
```

Salida esperada:

```
tcp   LISTEN 0      4096         *:3000            *:*    users:(("grafana",pid=14520,fd=12))
```

---

3. Primer Acceso y Configuración Inicial

🌐 Conexión vía navegador

Abre tu navegador web y dirígete a:

```
http://localhost:3000   (o http://<IP_DE_TU_SERVIDOR>:3000)
```

🔑 Credenciales predeterminadas:

- **Usuario:** `admin`
- **Contraseña:** `admin`

Pantalla de inicio de sesión de Grafana:
(Introduce admin / admin en el primer acceso)

🔒 Cambio forzoso de contraseña

En el primer inicio de sesión, Grafana solicitará obligatoriamente cambiar la contraseña por defecto. Establece una contraseña robusta (por ejemplo, `AdminSeguro2026!`).

---

4. Archivo de Configuración Principal (`grafana.ini`)

El comportamiento global de Grafana se define en el archivo `/etc/grafana/grafana.ini`.

```
sudo nano /etc/grafana/grafana.ini
```

Parámetros más relevantes:

- `[server] http_port = 3000` : Puerto TCP donde escucha la interfaz web.
- `[server] domain = localhost` : Nombre de dominio asignado al servidor.
- `[security] admin_user = admin` : Nombre del usuario administrador principal.
- `[smtp]` : Parámetros de servidor de correo para envío de alertas (se configurará en el Tema 7).

> ℹ️ **NOTE:** > Cualquier modificación en `/etc/grafana/grafana.ini` requiere reiniciar el demonio para surtir efecto: > ```bash > sudo systemctl restart grafana-server > ```

🔌 03. Añadir Prometheus como Data Source en Grafana

📋 Introducción

Un **Data Source** (origen de datos) es la conexión que permite a Grafana consultar un backend de almacenamiento. Para que Grafana pueda renderizar gráficos con las métricas del sistema, es necesario vincularlo con la API HTTP de Prometheus.

En esta unidad configuraremos Prometheus como Data Source predeterminado tanto desde la **interfaz gráfica** como mediante **aprovisionamiento declarativo YAML** (*Infrastructure as Code*).

---

1. Configuración interactiva desde la Interfaz Web

Paso 1: Acceder al menú de Data Sources

1. Inicia sesión en Grafana (`http://localhost:3000`).
2. En el menú lateral izquierdo, haz clic en **Connections** $\rightarrow$ **Data sources** (o navega al icono de engranaje **Configuration** $\rightarrow$ **Data Sources** según la versión de Grafana).
3. Haz clic en el botón azul **Add data source**.

Paso 2: Seleccionar el conector de Prometheus

1. En la lista de orígenes soportados, selecciona **Prometheus** (suele aparecer destacado en la categoría *Time series databases*).

Paso 3: Rellenar los parámetros de conexión

Configura los campos obligatorios:

Campo	Valor Recomendado	Explicación
Name	`Prometheus`	Nombre identificador del Data Source en Grafana.
Default	Activado (`Toggle On`)	Establece Prometheus como origen predeterminado para nuevos paneles.
Prometheus server URL	`http://localhost:9090`	Dirección y puerto donde escucha la API de Prometheus (si están en hosts distintos, indica `http://<IP_PROMETHEUS>:9090`).
HTTP Method	`POST`	Método de envío de peticiones (el método `POST` soporta queries más largas y complejas sin desbordar cabeceras).
Scrape interval	`15s`	Debe coincidir con el `scrape_interval` configurado en `prometheus.yml` .

> ⚠️ **WARNING:** > No utilices `localhost` si Grafana se está ejecutando dentro de un contenedor Docker aislado de la red del host; en ese caso debes usar la IP de la interfaz puente o el nombre del servicio en la red de Docker.

Paso 4: Validar la conexión (*Save & Test*)

1. Desplázate hasta la parte inferior de la página.
2. Haz clic en el botón verde **Save & test**.
3. Deberá aparecer un mensaje de confirmación con fondo verde:

```
✅ Successfully queried the Prometheus API.
```

Si aparece un error rojo de conexión: - Comprueba que Prometheus sigue en ejecución (`systemctl status prometheus`). - Verifica que el puerto 9090 no esté bloqueado por el firewall `ufw` (`sudo ufw allow 9090/tcp`).

---

2. Verificación de Métricas con la herramienta *Explore*

Grafana incluye una consola interactiva llamada **Explore** diseñada para depurar y probar consultas PromQL sin necesidad de crear un dashboard definitivo:

1. En el menú lateral izquierdo, haz clic en **Explore** (icono de brújula 🧭).
2. Asegúrate de que el desplegable superior tenga seleccionado el origen **Prometheus**.
3. En el campo de consulta (modo *Code* o *Builder*), introduce: `promql up`
4. Haz clic en **Run query** (arriba a la derecha).
5. Observarás una gráfica de líneas y una tabla inferior mostrando las instancias scrapeadas con valor `1` (operativa).

---

3. Método DevOps: Aprovisionamiento Automático con YAML

En entornos automatizados (Ansible, Puppet o Docker), Grafana permite autoconfigurar sus Data Sources al arrancar mediante archivos YAML en `/etc/grafana/provisioning/datasources/` :

```
sudo nano /etc/grafana/provisioning/datasources/prometheus.yaml
```

Contenido del archivo:

```
apiVersion: 1

datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://localhost:9090
    isDefault: true
    jsonData:
      httpMethod: POST
      timeInterval: 15s
    editable: true
```

Tras guardar el archivo, reinicia Grafana:

```
sudo systemctl restart grafana-server
```

El Data Source aparecerá creado y listo para usar sin requerir ninguna acción manual en la interfaz web.

🌐 04. Monitorización de Varios Servidores con Node Exporter

📋 Introducción

En una infraestructura real, el servidor de monitorización central (**Prometheus + Grafana**) supervisa decenas o cientos de máquinas virtuales o físicas distribuidas.

Para ello, se instala el agente **Node Exporter** en cada uno de los servidores cliente y se configuran los respectivos **targets** en el archivo `prometheus.yml` del servidor central.

🏗️ Topología del Laboratorio

```
[Diagrama de flujo de arquitectura disponible en la versión web de GitHub]
```

1. Instalación de Node Exporter en los Servidores Cliente

En **cada uno de los servidores** que desees monitorizar, realiza el siguiente procedimiento:

Método A: Instalación rápida mediante APT

```
sudo apt update
sudo apt install -y prometheus-node-exporter
```

El servicio arrancará automáticamente en el puerto `9100`.

Método B: Instalación con Binario Oficial de GitHub

```
# Crear usuario sin login
sudo useradd -rs /bin/false node_exporter

# Descargar e instalar la versión estable
cd /tmp
wget https://github.com/prometheus/node_exporter/releases/download/v1.7.0/node_exporter-1.7.0.linux-amd64.tar.gz
tar -xvf node_exporter-1.7.0.linux-amd64.tar.gz
sudo cp node_exporter-1.7.0.linux-amd64/node_exporter /usr/local/bin/

# Crear servicio systemd
sudo nano /etc/systemd/system/node_exporter.service
```

Añadir el contenido del servicio:

```
[Unit]
Description=Prometheus Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target
```

Iniciar y habilitar el servicio:

```
sudo systemctl daemon-reload
sudo systemctl enable --now node_exporter
```

Paso 2: Verificar apertura de puertos y endpoint de métricas

Verifica que el puerto `9100` está a la escucha en el nodo:

```
ss -plunt | grep 9100
```

Si el servidor tiene el cortafuegos `ufw` activado, permite el tráfico en el puerto `9100` exclusivamente desde la IP del servidor Prometheus:

```
sudo ufw allow from 172.26.12.21 to any port 9100 proto tcp
```

Comprueba desde el servidor central que las métricas son accesibles mediante `curl`:

```
curl http://<IP_DEL_NODO_CLIENTE>:9100/metrics | head -n 20
```

---

2. Configurar Prometheus para Recolectar Múltiples Servidores

En el **servidor central de Prometheus**, editamos el archivo `/etc/prometheus/prometheus.yml`:

```
sudo nano /etc/prometheus/prometheus.yml
```

Localizamos el bloque `scrape_configs` y definimos un job para los servidores Linux:

```
scrape_configs:
  # Monitorización del propio servidor Prometheus
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  # Monitorización de la granja de servidores Linux
  - job_name: "servidores_linux"
    scrape_interval: 10s
    static_configs:
      # Nodo 1: Servidor Local
      - targets: ["172.26.12.21:9100"]
        labels:
          servidor: "srv-monitoreo"
          entorno: "produccion"
          datacenter: "local"

      # Nodo 2: Servidor Remoto
      - targets: ["172.26.20.3:9100"]
        labels:
          servidor: "srv-profesor"
          entorno: "laboratorio"
          datacenter: "aula-213"

      # Nodo 3: Servidor Web Adicional
      - targets: ["192.168.1.50:9100"]
        labels:
          servidor: "srv-web"
          entorno: "desarrollo"
          datacenter: "cloud"
```

Recargar la configuración de Prometheus:

Recargamos la configuración sin reiniciar el servicio:

```
curl -X POST http://localhost:9090/-/reload
```

(O alternativamente: `sudo systemctl reload prometheus`)

---

3. Comprobar el Estado de los Targets en Prometheus

1. Abre en tu navegador la web de Prometheus: `http://localhost:9090`.
2. Dirígete a **Status** $\rightarrow$ **Targets**.
3. Deberás ver el grupo **`servidores_linux`** con todas las instancias añadidas:
4. El estado debe mostrarse en color verde: **`UP`**.
5. Se mostrarán las etiquetas personalizadas (`servidor="srv-profesor"`, `entorno="laboratorio"`, etc.).

---

4. Visualización en Grafana con el Dashboard "Node Exporter Full"

En lugar de construir decenas de paneles manualmente para cada servidor, Grafana permite importar plantillas comunitarias completas:

Paso 1: Importar el Dashboard ID 1860

1. Accede a Grafana (`http://localhost:3000`).
2. En el menú superior o lateral, haz clic en **Dashboards** $\rightarrow$ **New** $\rightarrow$ **Import**.
3. En el campo *Find via grafana.com*, introduce el identificador: `text 1860`
4. Haz clic en **Load**.
5. En el menú desplegable inferior, selecciona tu Data Source (**Prometheus**).
6. Haz clic en **Import**.

Paso 2: Filtrado dinámico entre servidores

El dashboard importado incluye automáticamente una barra superior con la variable `Host` / `Instance`: - Al hacer clic sobre el desplegable de servidores, puedes seleccionar **Nodo 1**, **Nodo 2** o **Todos**, visualizando al instante: - Carga de CPU por núcleo. - Memoria RAM disponible y Swap. - Tráfico por interfaz de red. - Espacio y operaciones I/O en disco.

🔥 05. Estresando al Servidor: Pruebas de Carga y Monitorización

📋 Objetivo de la Práctica

El propósito de someter un servidor a pruebas de estrés controladas es comprobar la **reactividad y precisión** de la pila de monitorización. Generaremos sobrecargas sintéticas de **CPU**, **memoria RAM** y **operaciones de entrada/salida (I/O) en disco**, observando cómo Prometheus registra los picos en sus series temporales y cómo Grafana actualiza los gráficos en tiempo real.

🛠️ Herramientas de Estrés: `stress` y `stress-ng`

Instalamos ambas utilidades en la máquina a someter a carga:

```
sudo apt update
sudo apt install -y stress stress-ng
```

1. Prueba 1: Estrés de CPU al 100%

Comando de ejecución:

```
# Somete a estrés 4 núcleos de CPU durante 60 segundos
stress --cpu 4 --timeout 60s
```

- `--cpu 4` : Lanza 4 procesos concurrentes que ejecutan cálculos continuos de raíces cuadradas (`sqrt()`) para saturar los núcleos de procesamiento.
- `--timeout 60s` : Detiene automáticamente la prueba transcurrido 1 minuto para evitar bloqueos del sistema.

🔍 Detección en Prometheus (PromQL):

En la pestaña *Graph* de Prometheus o en Grafana, ejecuta la siguiente consulta:

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[30s])) * 100)
```

📈 Comportamiento en Grafana:

- El gráfico de uso de CPU pasará de un estado habitual en reposo ($1\% - 5\%$) a una línea vertical que alcanza el **100%**.
- La métrica de carga media a 1 minuto (`node_load1`) comenzará a subir por encima del número de núcleos disponibles.

2. Prueba 2: Estrés de Memoria RAM

Comando de ejecución:

```
# Asigna y libera continuamente 2 GB de memoria RAM durante 60 segundos
stress --vm 2 --vm-bytes 1G --timeout 60s
```

- `--vm 2` : Inicia 2 trabajadores dedicados a la gestión de memoria virtual.
- `--vm-bytes 1G` : Cada trabajador asigna 1 GB de memoria mediante llamadas `malloc()` y escribe en ella continuamente.

🔍 Detección en Prometheus (PromQL):

```
# Memoria RAM ocupada en Megabytes
(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / 1024 / 1024
```

📈 Comportamiento en Grafana:

- La gráfica de **Memoria Disponible** (*Available Memory*) experimenta una caída abrupta de 2 GB.
- Si el servidor tiene menos RAM física de la solicitada, se observa en tiempo real cómo el kernel recurre a la partición de intercambio (**Swap In / Swap Out**).

3. Prueba 3: Estrés de Entrada/Salida en Disco (I/O)

Comando de ejecución:

```
# Genera operaciones masivas de sincronización de disco (sync) durante 60 segundos
stress --io 4 --hdd 2 --timeout 60s
```

- `--io 4` : Lanza 4 procesos ejecutando llamadas continuas a `sync()` .
- `--hdd 2` : Lanza 2 trabajadores escribiendo y leyendo bloques masivos en disco con la llamada `write()` .

🔍 Detección en Prometheus (PromQL):

```
# Tasa de escritura en disco en Megabytes por segundo
rate(node_disk_written_bytes_total[1m]) / 1024 / 1024
```

📈 Comportamiento en Grafana:

- El panel de **Disk IOPS** y **Disk Throughput** muestra un disparo inmediato en la tasa de transferencia (MB/s).
- La métrica de latencia o tiempo de espera de disco (`r_await` / `w_await`) aumenta significativamente.

---

4. Prueba 4: Estrés Masivo Combinado con `stress-ng`

Para simular una situación extrema de degradación de servicio en un servidor de producción:

```
stress-ng --cpu 2 --vm 1 --vm-bytes 512M --hdd 1 --timeout 45s --metrics-brief
```

La opción `--metrics-brief` genera un informe estadístico resumido al finalizar la prueba:

```
stress-ng: info:  [21540] dispatching processes: 2 cpu, 1 vm, 1 hdd
stress-ng: info:  [21540] successful run completed in 45.02s
stress-ng: info:  [21540] stressor       bogo ops real time  usr time  sys time
stress-ng: info:  [21540] cpu              48201     45.00     89.12      0.15
stress-ng: info:  [21540] vm              124050     45.01     12.30     32.40
stress-ng: info:  [21540] hdd              23100     45.01      0.80     28.10
```

---

5. Cuadro Comparativo de Métricas Monitorizadas

Tipo de Estrés	Métrica Clave en Prometheus	Umbral de Alerta Típico
CPU	`node_cpu_seconds_total` / `node_load1`	$> 85\%$ durante 2 min
Memoria	`node_memory_MemAvailable_bytes`	$< 10\%$ de la RAM total
Disco I/O	`node_disk_io_time_seconds_total`	Utilización disco $> 95\%$
Filesystem	`node_filesystem_free_bytes`	Espacio libre $< 15\%$

> 💡 **TIP:** > Durante estas pruebas, reduce temporalmente el `scrape_interval` de tu job en `prometheus.yml` a `5s` para apreciar con mayor granularidad la curva de aceleración y enfriamiento de los recursos del sistema.

📊 06. Creación de Consultas PromQL y Diseño de Nuevos Paneles

📋 Introducción

Dominar **PromQL** (*Prometheus Query Language*) es la habilidad esencial de cualquier administrador de sistemas que gestione Prometheus. Permite extraer valor de millones de puntos de datos en bruto mediante agregaciones, filtros dimensionales y cálculos de tasas de cambio en tiempo real.

En esta unidad desglosaremos la sintaxis de PromQL y construiremos un **dashboard personalizado desde cero** en Grafana con visualizaciones de tipo *Time series*, *Gauge*, *Stat* y variables dinámicas.

1. Fundamentos de PromQL

🔹 Tipos de Vectores

1. **Instant Vector (Vector instantáneo):** Muestra el valor más reciente de cada serie temporal en un momento concreto. `promql node_memory_MemAvailable_bytes`
2. **Range Vector (Vector de rango):** Muestra un conjunto de valores históricos a lo largo de una ventana de tiempo (se indica entre corchetes `[ventana]`). `promql node_cpu_seconds_total[5m]`

🔹 Funciones Esenciales

Función	Utilidad	Ejemplo
`rate()`	Calcula la tasa de incremento medio por segundo de un contador a lo largo de una ventana de tiempo. Ideal para métricas acumulativas continuas.	`rate(node_network_receive_bytes_total[1m])`
`irate()`	Calcula la tasa instantánea considerando exclusivamente los **dos últimos puntos** de la serie temporal. Más reactivo para picos breves.	`irate(node_cpu_seconds_total[1m])`
`sum()`	Suma los valores de las series coincidentes.	`sum(node_cpu_seconds_total)`
`avg()`	Calcula el promedio de las series temporales.	`avg(rate(node_cpu_seconds_total[1m]))`
`by (...)`	Cláusula de agrupación: preserva únicamente las etiquetas indicadas en el resultado agregado.	`avg by (instance) (...)`
`without (...)`	Cláusula de exclusión: elimina las etiquetas indicadas del resultado.	`sum without (cpu) (...)`

2. Consultas PromQL Clave para Administración de Servidores

1. Porcentaje Real de Uso de CPU Global

Los sistemas Linux reportan el tiempo de CPU acumulado en segundos por cada modo (`idle`, `user`, `system`, `iowait`, etc.). Para calcular el uso real, restamos el porcentaje de tiempo inactivo (*idle*) al 100%:

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
```

2. Porcentaje de Memoria RAM Ocupada

La memoria RAM ocupada real se obtiene restando la memoria disponible a la memoria total:

```
100 * (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes))
```

3. Porcentaje de Espacio Libre en el Disco Raíz (`/`)

```
100 * (node_filesystem_free_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"})
```

4. Tráfico de Red Entrante y Saliente (en KB/s)

```
# Tráfico Recibido (Download)
rate(node_network_receive_bytes_total{device="eno1"}[1m]) / 1024

# Tráfico Transmitido (Upload)
rate(node_network_transmit_bytes_total{device="eno1"}[1m]) / 1024
```

5. Estado de Disponibilidad de Servidores (`UP / DOWN`)

```
up{job="servidores_linux"}
```

(Devuelve `1` si el target responde al scrape o `0` si el host o el exporter están caídos).

---

3. Diseño de un Dashboard Personalizado en Grafana

Vamos a diseñar un cuadro de mando profesional que reúna estas métricas en un único panel unificado:

```
[Diagrama de flujo de arquitectura disponible en la versión web de GitHub]
```

Paso 1: Crear el Dashboard

1. En Grafana, haz clic en **Dashboards** $\rightarrow$ **New** $\rightarrow$ **New Dashboard**.
2. Haz clic en **+ Add visualization**.
3. Selecciona el origen de datos **Prometheus**.

Paso 2: Crear el Panel de Estado (*Stat*)

- **Query (A):** `promql up{instance=~"$instance"}`
- **Panel options:**
- *Title:* `Estado del Servidor`
- *Type:* **Stat**
- **Value mappings (Mapeo de valores):**
- Valor `1` $\rightarrow$ Texto: `ONLINE` , Color: Verde.
- Valor `0` $\rightarrow$ Texto: `OFFLINE` , Color: Rojo.

Paso 3: Crear el Panel de Uso de RAM (*Gauge*)

- **Query (A):** `promql 100 * (1 - (node_memory_MemAvailable_bytes{instance=~"$instance"} / node_memory_MemTotal_bytes{instance=~"$instance"}))`
- **Panel options:**
- *Title:* `Consumo de Memoria RAM`
- *Type:* **Gauge**
- **Standard options:**
- *Unit:* `Misc` $\rightarrow$ `Percent (0-100)`
- *Min:* `0` , *Max:* `100`
- **Thresholds (Umbrales de color):**
- Verde: Base (0%).
- Amarillo: $> 75\%$.
- Rojo: $> 90\%$.

Paso 4: Crear el Panel de Histórico de CPU (*Time Series*)

- **Query (A):** `promql 100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle", instance=~"$instance"}[1m])) * 100)`
- **Panel options:**
- *Title:* `Evolución de Carga de CPU`
- *Type:* **Time series**
- **Standard options:**
- *Unit:* `Misc` $\rightarrow$ `Percent (0-100)`
- *Min:* `0` , *Max:* `100`

Paso 5: Crear el Panel de Tráfico de Red (*Time Series*)

- **Query (A) - Recepción (RX):** `promql rate(node_network_receive_bytes_total{instance=~"$instance", device!~"lo|docker.*|veth.*"}[1m])` *(Legend: `RX {{device}}`)*
- **Query (B) - Transmisión (TX):** `promql rate(node_network_transmit_bytes_total{instance=~"$instance", device!~"lo|docker.*|veth.*"}[1m])` *(Legend: `TX {{device}}`)*
- **Standard options:**
- *Unit:* `Data rate` $\rightarrow$ `Bytes/sec (SI)`

---

4. Creación de Variables Dinámicas (Plantillas de Servidores)

Para que el dashboard permita seleccionar cualquier servidor monitorizado con un desplegable:

1. Dentro de tu Dashboard, haz clic en el icono de engranaje **Dashboard settings** (arriba a la derecha).
2. Haz clic en **Variables** $\rightarrow$ **Add variable**.
3. Configura los siguientes parámetros:
4. **Name:** `instance`
5. **Label:** `Servidor`

6. **Type:** `Query`
7. **Data source:** `Prometheus`
8. **Query:** `promql label_values(up{job="servidores_linux"}, instance)`
9. **Selection options:**
    - Activa: `Multi-value` (Permite seleccionar varios servidores a la vez).
    - Activa: `Include All option` (Permite ver todos a la vez).
10. Haz clic en **Apply** y guarda el Dashboard.

> 💡 **TIP:** > A partir de este momento, cualquier consulta que utilice `{instance=~"$instance"}` responderá de forma automática al servidor seleccionado en la barra superior.

🔔 07. Configuración de Alertas con Gmail

📋 Introducción

La monitorización reactiva no es suficiente: los administradores no pueden vigilar las pantallas de Grafana de forma ininterrumpida. El sistema debe ser **proactivo** y enviar notificaciones inmediatas por correo electrónico cuando se detecte una incidencia crítica (caída de un servicio, saturación de CPU o falta de espacio en disco).

En esta unidad configuraremos el motor **Grafana Alerting** junto al servidor SMTP de **Google (Gmail)** mediante una contraseña de aplicación segura.

🏗️ Flujo de Alertas

```
[Diagrama de flujo de arquitectura disponible en la versión web de GitHub]
```

1. Generar una Contraseña de Aplicación en Google

Por motivos de seguridad, Google no permite utilizar la contraseña habitual de tu cuenta de correo en aplicaciones de terceros:

1. Inicia sesión en tu cuenta de Google y accede a Mi Cuenta Google - Seguridad.
2. Asegúrate de tener activada la **Verificación en 2 pasos**.
3. En la barra de búsqueda de la cuenta de Google, escribe **"Contraseñas de aplicaciones"** (*App Passwords*).
4. Asigna un nombre a la aplicación (por ejemplo, `Grafana-Alertas`).
5. Haz clic en **Crear**.
6. Google generará un código único de **16 caracteres** (ej. `abcd efgh ijkl mnop`).
7. **Copia y guarda esta contraseña**, ya que no volverá a mostrarse.

2. Configurar el Servidor SMTP en Grafana (`grafana.ini`)

Editamos el archivo de configuración del servidor Grafana:

```
sudo nano /etc/grafana/grafana.ini
```

Buscamos la sección `[smtp]` (alrededor de la línea 650) y ajustamos los parámetros eliminando los puntos y coma `;` al inicio de cada línea:

```
[smtp]
enabled = true
host = smtp.gmail.com:587
user = tu_correo@gmail.com
# Introduce la contraseña de aplicación de 16 caracteres generada en Google
password = abcd efgh ijkl mnop
cert_file =
key_file =
skip_verify = false
from_address = tu_correo@gmail.com
from_name = Grafana Monitor ASIR
ehlo_identity =
startTLS_policy = OpportunisticStartTLS
```

Guardamos los cambios (`Ctrl+O`, `Enter`, `Ctrl+X`) y reiniciamos el servicio:

```
sudo systemctl restart grafana-server
```

3. Crear el Punto de Contacto (*Contact Point*) en Grafana

1. Inicia sesión en Grafana (`http://localhost:3000`).
2. En el menú lateral, dirígete a **Alerting** $\rightarrow$ **Contact points**.
3. Haz clic en **+ Add contact point**.
4. Rellena los datos:
5. **Name:** `Admin-Gmail`
6. **Integration:** `Email`
7. **Addresses:** El correo electrónico donde deseas recibir las alertas (ej. `tu_correo@gmail.com`).
8. Antes de guardar, haz clic en el botón **Test contact point**.
9. Haz clic en **Send test notification**.
10. **Revisa tu bandeja de entrada de Gmail:** Deberás haber recibido un correo con el asunto `[Test] Test notification` procedente de *Grafana Monitor ASIR*.
11. Haz clic en **Save contact point**.

4. Definir la Regla de Alerta (*Alert Rule*)

Vamos a programar una alerta que se active si el uso de CPU de cualquier servidor supera el **85%** de forma continua durante más de **1 minuto**:

1. En Grafana, ve a **Alerting** $\rightarrow$ **Alert rules** $\rightarrow$ **+ New alert rule**.
2. **Paso 1: Nombre de la regla**
3. *Rule name:* `Alerta_CPU_Sobrecarga`
4. **Paso 2: Definir la consulta y condición**
5. *Data source:* `Prometheus`
6. *Query (A):* `promql 100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)`
7. *Expression (B) - Reduce:* Función `Last`, Input `A`.
8. *Expression (C) - Threshold:* Input `B`, Condición: `IS ABOVE 85`.
9. **Paso 3: Criterios de evaluación**
10. *Evaluate every:* `30s` (evalúa la condición cada 30 segundos).
11. *For:* `1m` (la condición debe mantenerse activa durante 1 minuto completo para pasar de `Pending` a `Firing`).
12. **Paso 4: Etiquetas y detalles de la notificación**
13. Añade la etiqueta: `severity = critical`
14. *Summary:* `Consumo de CPU crítico en el servidor {{ $labels.instance }}`
15. *Description:* `La CPU ha superado el 85% de uso continuo durante más de 1 minuto.`
16. Haz clic en **Save rule and exit** (arriba a la derecha).

---

5. Configurar la Política de Notificaciones (*Notification Policy*)

Para garantizar que las alertas críticas se envíen al contacto de Gmail:

1. Ve a **Alerting** $\rightarrow$ **Notification policies**.
2. En la política raíz (*Root policy*), edita el contacto predeterminado y selecciona **`Admin-Gmail`**.
3. Guarda la política (*Save policy*).

---

6. Simulación Práctica: Disparo y Resolución de la Alerta

Vamos a estresar intencionadamente el servidor para verificar todo el ciclo de vida de la alerta:

1. Provocar la sobrecarga de CPU:

En el servidor monitorizado, ejecutamos una carga al 100% durante 2 minutos:

```
stress --cpu 4 --timeout 120s
```

2. Ciclo de estados en Grafana Alerting:

- **0 a 30s:** La CPU sube al 100%. La regla pasa al estado amarillo **`Pending`** (condición cumplida, esperando tiempo de persistencia).
- **A partir de 60s:** Se supera el tiempo configurado (`For 1m`). La regla pasa al estado rojo **`Firing`**.
- **Notificación:** Grafana envía de inmediato el correo a Gmail con el detalle del servidor y la gráfica asociada.

3. Recuperación automática (*Resolved*):

- Al concluir los 120 segundos de `stress`, la CPU vuelve al reposo (2%).
- Grafana detecta que el valor ha caído por debajo de 85 y la regla pasa a estado **`Normal`**.
- Se envía automáticamente un correo de resolución con el encabezado **`[RESOLVED]`** confirmando que la incidencia ha sido solventada.

🗄️ 08. Tarea Práctica: Monitorización de Bases de Datos MariaDB

🎯 Objetivo de la Tarea

En los entornos corporativos, las bases de datos relacionales constituyen el núcleo crítico de cualquier servicio. Una saturación en el número de conexiones o un aumento descontrolado de consultas lentas (*slow queries*) puede degradar por completo las aplicaciones dependientes.

En esta tarea práctica desplegaremos una solución integral de monitorización para **MariaDB** utilizando **Prometheus**, el agente especializado `mysqld_exporter` y **Grafana**.

🏗️ Arquitectura de la Solución

```
[Diagrama de flujo de arquitectura disponible en la versión web de GitHub]
```

Paso 1: Instalación y Preparación de MariaDB Server

Si el servidor no cuenta con MariaDB instalado, lo instalamos e iniciamos:

```
sudo apt update
sudo apt install -y mariadb-server

# Habilitar e iniciar el servicio
sudo systemctl enable --now mariadb

# Comprobar que está activo y escuchando en el puerto 3306
sudo systemctl status mariadb
ss -plunt | grep 3306
```

Paso 2: Creación del Usuario de Monitorización en MariaDB

> ⚠️ **IMPORTANT:** > **Principio de Mínimo Privilegio:** Nunca debe utilizarse el usuario administrador `root` para alimentar un exporter de métricas. Se debe crear un usuario de sistema dedicado con permisos estrictamente de solo lectura y limitar su número máximo de conexiones concurrentes para no penalizar a la base de datos.

Accedemos a la consola de MariaDB como superusuario:

```
sudo mysql -u root
```

Ejecutamos las siguientes sentencias SQL:

```
-- 1. Crear el usuario mysqld_exporter limitando sus conexiones concurrentes a 3
CREATE USER 'mysqld_exporter'@'localhost' IDENTIFIED BY 'PasswordSeguro123!' WITH MAX_USER_CONNECTIONS 3;

-- 2. Conceder los permisos de lectura de estados y variables globales del motor
GRANT PROCESS, REPLICATION CLIENT, SELECT ON *.* TO 'mysqld_exporter'@'localhost';

-- 3. Aplicar los privilegios
FLUSH PRIVILEGES;

-- 4. Salir de la consola
EXIT;
```

Paso 3: Instalación y Configuración de `mysqld_exporter`

1. Creación de usuario de sistema y descarga del binario

```
# Crear usuario de sistema dedicado
sudo useradd -rs /bin/false mysqld_exporter

# Descargar e instalar el binario oficial
cd /tmp
wget https://github.com/prometheus/mysqld_exporter/releases/download/v0.15.1/mysqld_exporter-0.15.1.linux-amd64.tar.gz
tar -xvf mysqld_exporter-0.15.1.linux-amd64.tar.gz
sudo cp mysqld_exporter-0.15.1.linux-amd64/mysqld_exporter /usr/local/bin/
```

2. Archivo de credenciales protegido (`.my.cnf`)

Creamos un directorio seguro y almacenamos las credenciales de conexión:

```
sudo mkdir -p /etc/mysqld_exporter
sudo nano /etc/mysqld_exporter/.my.cnf
```

Introducimos la configuración de acceso local:

```
[client]
user=mysqld_exporter
password=PasswordSeguro123!
host=localhost
port=3306
```

Aseguramos los permisos para que **únicamente** el usuario `mysqld_exporter` pueda leer el archivo:

```
sudo chown -R mysqld_exporter:mysqld_exporter /etc/mysqld_exporter
sudo chmod 600 /etc/mysqld_exporter/.my.cnf
```

---

3. Creación del servicio Systemd

Creamos la unidad de servicio para gestionar el exporter en segundo plano:

```
sudo nano /etc/systemd/system/mysqld_exporter.service
```

Insertamos la siguiente definición:

```
[Unit]
Description=Prometheus MySQL / MariaDB Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=mysqld_exporter
Group=mysqld_exporter
Type=simple
ExecStart=/usr/local/bin/mysqld_exporter \
    --config.my-cnf=/etc/mysqld_exporter/.my.cnf \
    --web.listen-address=0.0.0.0:9104

Restart=always

[Install]
WantedBy=multi-user.target
```

Iniciamos y habilitamos el servicio:

```
sudo systemctl daemon-reload
sudo systemctl enable --now mysqld_exporter

# Verificar estado y puerto 9104
sudo systemctl status mysqld_exporter
ss -plunt | grep 9104
```

---

4. Verificar la recolección de métricas

Comprobamos que el exporter está extrayendo datos reales de MariaDB:

```
curl http://localhost:9104/metrics | grep mysql_global_status_threads_connected
```

(Deberá responder con un valor numérico indicando las conexiones abiertas actualmente).

Paso 4: Integración en Prometheus (`prometheus.yml`)

En el servidor Prometheus, editamos el archivo `/etc/prometheus/prometheus.yml` :

```
sudo nano /etc/prometheus/prometheus.yml
```

Añadimos el nuevo job de recolección al final del bloque `scrape_configs` :

```
  # Monitorización de la base de datos MariaDB
  - job_name: "mariadb_database"
    scrape_interval: 10s
    static_configs:
      - targets: ["localhost:9104"]
        labels:
          servicio: "mariadb"
          rol: "database-server"
          entorno: "produccion"
```

Recargamos la configuración de Prometheus:

```
curl -X POST http://localhost:9090/-/reload
```

Comprobamos en `http://localhost:9090/targets` que el target **`mariadb_database`** aparece en estado verde **`UP`**.

Paso 5: Visualización en Grafana con Dashboard de MariaDB

Para visualizar las métricas relacionales de MariaDB importamos un dashboard comunitario especializado:

1. Inicia sesión en Grafana (`http://localhost:3000`).
2. Ve a **Dashboards** $\rightarrow$ **New** $\rightarrow$ **Import**.
3. En el campo de ID introduce: `text 7362` *(Alternativa oficial: ID `6844` - MySQL Overview).*
4. Selecciona tu Data Source **Prometheus** y haz clic en **Import**.

📊 Métricas Clave Monitorizadas en el Panel:

Métrica	PromQL	Significado
Conexiones Activas	`mysql_global_status_threads_connected`	Conexiones de clientes abiertas actualmente.
Consultas por Segundo (QPS)	`rate(mysql_global_status_queries[1m])`	Caudal de transacciones que atiende el servidor.
Consultas Lentas (Slow Queries)	`rate(mysql_global_status_slow_queries[5m])`	Consultas que sobrepasan el tiempo de `long_query_time`.
InnoDB Buffer Pool Hit Ratio	`rate(mysql_global_status_innodb_buffer_pool_reads[1m])`	Eficiencia de lectura de páginas en memoria RAM vs disco.
Tráfico de Red SQL	`rate(mysql_global_status_bytes_sent[1m])`	Ancho de banda saliente generado por consultas de datos.

Paso 6: Prueba de Carga y Estrés con `mysqlslap`

Para validar que los paneles y alertas responden de forma realista, utilizaremos la herramienta oficial de benchmarking **`mysqlslap`** incluida con MariaDB:

```
# Simula 30 conexiones simultáneas realizando 10 iteraciones de consultas de prueba
mysqlslap --user=root -p --concurrency=30 --iterations=10 --number-int-cols=5 --number-char-cols=20 --auto-generate-sql
```

📈 Observación en tiempo real en Grafana:

- El velocímetro de **Threads Connected** salta de 2 a 30 conexiones.
- La gráfica de **Questions / Queries per Second** experimenta un pico inmediato de miles de consultas concurrentes.
- El uso de memoria del **InnoDB Buffer Pool** se incrementa al cachear las tablas generadas sintéticamente.

🏁 Conclusión de la Tarea

Con la finalización de esta práctica, disponemos de una infraestructura integral de observabilidad: 1. **Prometheus** centralizando métricas de sistema operativo y motores de bases de datos. 2. **Node Exporter** y **mysqld_exporter** capturando métricas de bajo nivel y de servicio. 3. **Grafana** ofreciendo cuadros de mando intuitivos con métricas de salud y rendimiento. 4. **Sistema de Alertas** activo que notificará automáticamente a nuestro correo de Gmail ante cualquier anomalía.
